

# Project 1 —Dual ARMv8 with DMA2 Implementation Report

N26140804 張暉俊

## 1. What I Built

### 1.1 System Architecture

Following the instructor's posts in the class group and the **Virtualizer for Project 1.pdf** handout, I built the system starting from the Cortex-A35 Example shipped with Virtualizer. The original example has one cluster with four CPUs running dhrystone. I reduced `number_of_cpus` to 2, keeping only *ARMv8\_0* and *ARMv8\_1*, then wired DMA2 and four RAMs around them to form the topology in spec Figure 1. The four example UARTs are kept in place; *UART\_2* and *UART\_3* do not affect operation, while *UART\_0* and *UART\_1* still map to *ARMv8\_0* and *ARMv8\_1* respectively.

The complete wiring is shown in the Instance Connectivity Diagram. The middle block is *ARM0* (two Cortex-A35 cores), with the four RAMs on the upper right. *DMA2\_1* on the left routes its *INT0* and *INT1* interrupt lines to *ARM0*'s *nIRQ\_0* and *nIRQ\_1*, and *SharedmemoryMap* at the bottom connects to the four UARTs.

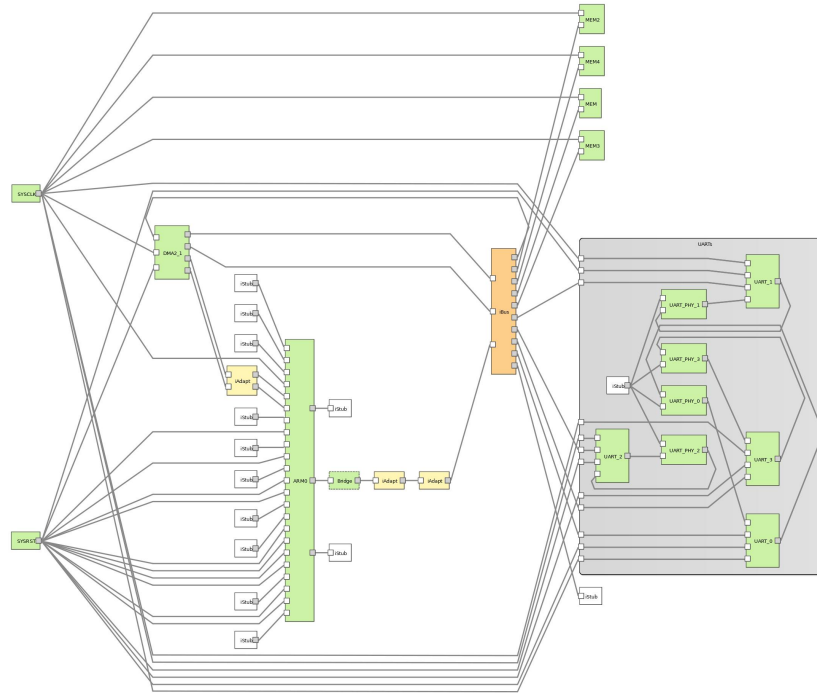


Figure 1: Instance Connectivity Diagram of the full system

### 1.2 Memory Map and DMA2 Register Layout

The whole SoC address space is managed by *SharedmemoryMap*. The allocation is as follows.

| Block                  | Address Range           | Size      | Purpose               |
|------------------------|-------------------------|-----------|-----------------------|
| <i>MEM (RAM0)</i>      | 0x00000000 ~ 0x07FFFFFF | 128 MB    | <i>ARMv8_0</i> local  |
| <i>MEM2 (RAM1)</i>     | 0x08000000 ~ 0x0FFFFFFF | 128 MB    | <i>ARMv8_1</i> local  |
| <i>DMA2_1.Socket</i>   | 0x10000000 ~ 0x1000001F | 32 B      | DMA2 control register |
| <i>MEM3 (RAM2)</i>     | 0x20000000 ~ 0x2FFFFFFF | 256 MB    | shared                |
| <i>MEM4 (RAM3)</i>     | 0x30000000 ~ 0x3FFFFFFF | 256 MB    | shared                |
| <i>UART_0 ~ UART_3</i> | 0x40000000 ~ 0x40003FFF | 4 KB each | UART register         |

Table 1: SoC memory map and DMA2 register allocation

DMA2 packs eight 4-byte registers into 32 bytes starting at 0x10000000, split into two sets. The first set (offsets 0x00 ~ 0x0C: SOURCE0, TARGET0, SIZE0, START0) is operated by *ARMv8\_0*; the second set (offsets 0x10 ~ 0x1C, mirror naming) by *ARMv8\_1*. Each set is serviced by its own SC\_CTHREAD inside the DMA2 module, so the two cores can issue DMA requests in parallel without interference.

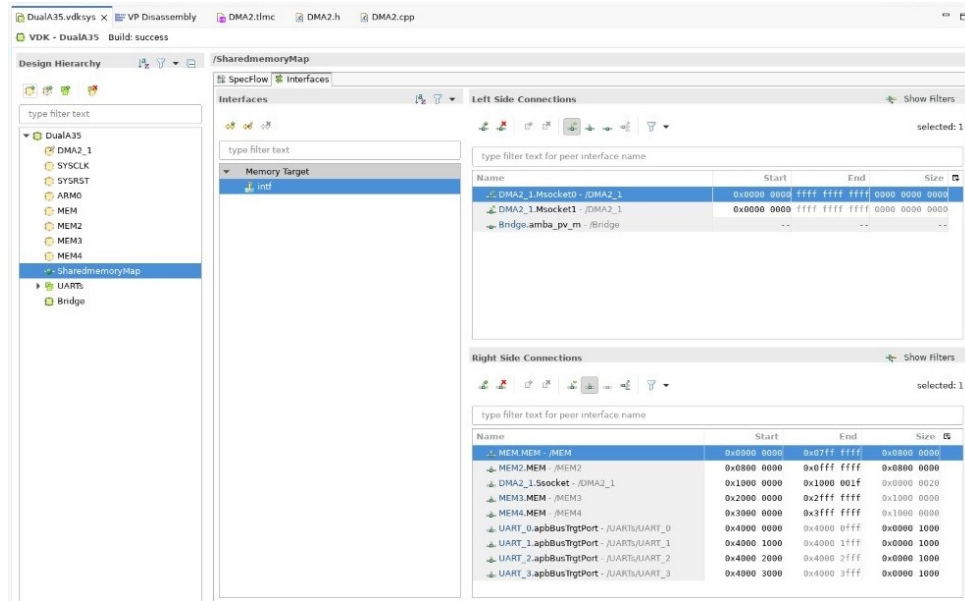


Figure 2: SharedmemoryMap address allocation

### 1.3 Bare-metal Data Exchange Flow

Per the spec, after boot *ARMv8\_0* fills 1KB of *RAM2* from 0x20000400 with repeated “0123456789ABCDEF”, while *ARMv8\_1* fills 1KB of *RAM3* from 0x30000400 with repeated “FEDCBA9876543210”. Both sides then enter the main loop and take turns via a sync flag at 0x20000800. One round:

1. *ARMv8\_0* waits for FLAG = 0, programs DMA channel 0 (SOURCE0, TARGET0, SIZE0, START0) for *RAM2* → *RAM3*, polls DMA2\_CH0\_SIZE until zero, writes CTRL0 = 0, then sets FLAG = 1.
2. *ARMv8\_1* waits for FLAG = 1, symmetrically operates channel 1 for *RAM3* → *RAM2*, and sets FLAG back to 0.

After three rounds both sides halt. The synchronization uses polling rather than IRQ; 2.3 explains why.

## 2. Roadblocks and How I Got Past Them

### 2.1 Using Disassembly to Get UART Output Working

The first time I ran the bare-metal code the console produced no output at all. I initially blamed `UART_BASE`, but realized the problem ran deeper after the built-in `dhrystone` elf printed fine in the same setup. That meant the model and the UART were OK, so the issue had to be in my own code. The instructor had suggested reading the source directly, but I could not locate the source for this example, so I disassembled `dhrystone_aa64_cpu0.axf` and traced what it did from reset to `printf`. That trail revealed three things I had not known.

1. Both CPUs start from `PC = 0x0`

I had assumed `cpu1` would start automatically from *RAM1*'s base `0x08000000`, but the disasm showed `dhrystone` reads `MPIDR_EL1` at the top and branches by `Aff0`. Both cores enter `0x0` with the same code, and the program needs a trampoline that routes `Aff0 == 1` to `0x08000000` so `cpu0` can run its own code undisturbed. The ARM architecture reference manual leaves this as “implementation-defined”; only disassembly reveals what the model actually chose.

2. The `vector_table` cannot sit at `0x0` directly

If it sits at `0x0`, the reset `PC = 0` lands on the first sync vector, which defaults to a `b . infinite loop`. The fix is to put `_start` in the `.vectors` section (the linker pins it to `0x0`), push the `vector_table` behind `_start` using `.balign 2048`, then use `msr vbar_el3, x0` inside `_start` to point the vector base to the new location. This ordering is barely mentioned in textbook ARMv8 chapters because it concerns how the model implements reset rather than what the ISA specifies.

3. The UART must be initialized before writes succeed

Disassembling `Setup_UART` confirmed the model uses PL011. Four registers must be written in order: `IBRD`, `FBRD`, `LCR_H`, `CR`. Among these, `CR = 0x301` (`UARTEN | TXE | RXE`) is the key to enabling both transmit and receive. Before figuring this out I had assumed writing `DR` straight after reset would print; nothing came out and the console stayed black.

Only after fixing all three did the UART finally start printing.

### 2.2 Getting DMA2 to Run Correctly

For DMA2 I started from the instructor's source (`Proj1DMA2Exp.tgz`). The original logic already covers two sets of control registers, two `SC_CTHREADS`, and the `INT0`, `INT1` interrupt outputs, so the overall flow did not need changing. Running it inside Virtualizer, however, surfaced three spots that needed fixing. Each is a pitfall where the code reads as reasonable but clashes with how Virtualizer, TLM-2.0, or SystemC actually behaves.

1. The `addr -= BASE`; at the start of `b_transport` must be removed

This line is correct inside a standalone testbench (e.g. `verifyDMA2` over `TLM2Bus`), because the testbench passes the full system address through and the slave must subtract `BASE` itself to obtain the register offset. But Virtualizer's `SharedmemoryMap` already strips the base offset before delivering the transaction to the slave socket; the incoming `addr` arrives as a register-relative offset in `0x00 ~ 0x1C`. Subtracting `BASE` a second time drops every case into the default `ADDRESS_ERROR` branch, and DMA receives no control register writes whatsoever.

2. `memcpy` cannot be written directly into registers like `SOURCE0`

The original code was `memcpy(&SOURCE0, r_data, r_len)`, which reads as completely reasonable. The catch is that registers like `SOURCE0` are of type `sc_uint<32>`, which derives from `sc_value_base` and contains virtual functions, making it non-POD. Its memory layout places a vtable pointer in the first 8 bytes, with the actual value `m_val` at a later offset. `memcpy` directly into the object's raw memory only clobbers the vtable region, leaving `m_val` permanently at 0, so `dma_process`

always reads `START` as 0 and the DMA never starts. The fix is to memcpy into a POD unsigned int first, then assign normally with `SOURCE0 = wval;`; the read path symmetrically goes through a POD intermediate.

3. The master socket must set `streaming_width` when issuing a transaction

`tlm_generic_payload` defaults `streaming_width` to 0. When Virtualizer's `SharedmemoryMap` sees 0 it rejects the transaction outright, prints `transaction streaming width is set to zero`, and aborts the simulation. The TLM-2.0 convention is that non-streaming transfers should set `streaming_width` equal to `data_length`; this is written quietly in the spec but the tool enforces it strictly. Once added, the master socket can read and write data normally.

Only after these three fixes did DMA2 actually work inside Virtualizer. Looking back, all three share the same pattern: the original source looks fine inside a standalone testbench, but on a full simulation platform the implicit requirements of TLM-2.0 and the memory layout of SystemC objects surface every pitfall the testbench never reached.

## 2.3 IRQ Doesn't Reach the CPU, Switching to Polling

Spec item 6 says “describe your interrupt synchronization implementation”, so my initial design used IRQ-based sync: once DMA completes it raises *nIRQ*, the CPU enters its IRQ handler to clear CTRL and set FLAG, then the main loop continues. In practice the ARM side never entered the handler at all.

DMA2.cpp's own `std::cout` prints `DMA INTO sent` and `DMA clears INTO`, so the interrupt signal inside the model is actually moving. But the ARM side gives no response. Reading the ARM Cortex-A35 TLM2 LT PSP manual eventually revealed that the `GICDISABLE` parameter defaults to true, and in that state Fast Model does not deliver the *nIRQ* signal to the CPU core.

Details

Simulation Output v Console Details UART0 UART1 UART2 UART3 UART7

Clear Scroll Lock Local Node

```

DMA received READ at 0x100000000 length: 4; 0xc
DMA read 0x02000007f; 0x3730354
DMA write 0x03000007f; 0x1730354
DMA read 0x03000007f; 0x4211930
DMA write 0x02000007f; 0x4211930
DMA read 0x03000007f; 0x4214443

UART2 SPI
DMA received READ at 0x100000000 length: 4; 0x0
DMA received WRITE at 0x100000000 length: 4; 0x00000000
DMA clears INT0

DMA received WRITE at 0x100000010 length: 4; 0x00000000
DMA received WRITE at 0x100000020 length: 4; 0x00000000
DMA received WRITE at 0x100000030 length: 4; 0x00000001
DMA read 0x030000040; 0x1040546
DMA write 0x020000000; 0x1040546
DMA read 0x030000040; 0x1039142
DMA read 0x030000040; 0x1039142
DMA write 0x020000000; 0x1039142
DMA read 0x030000040; 0x1031233
DMA write 0x020000000; 0x1031233
DMA read 0x030000040; 0x1020202

```

DMA2 side console prints *INT0* sent / clear,  
confirming the signal was fired

The screenshot shows the Visual Studio IDE with the 'Design Hierarchy' and 'Parameters' windows open. The 'Design Hierarchy' on the left shows a tree structure with 'Dual35-vdbyon' as the root, containing 'vSP', 'vSP-Dual35', and 'Build-success'. The 'vSP-Dual35' component is expanded, showing 'ARM9' as a sub-component. The 'Parameters' window on the right shows the 'Parameters' tab for the 'ARM9' component. It displays a table of parameters with columns for 'Name', 'Developer Value', and 'Baseline Value (the processor ARM9-spr.fg)'. The table contains one row for the 'config' parameter, which is set to 'DEENABLE'.

| Name   | Developer Value | Baseline Value (the processor ARM9-spr.fg) |
|--------|-----------------|--------------------------------------------|
| config | DEENABLE        | true                                       |

*ARM0* Parameters: GICDISABLE defaults to true, so  
Fast Model does not deliver *nIRQ*

Once GICDISABLE became the prime suspect, I tried several directions to get IRQ working. First I suspected the active-level setting; the PSP manual says *nIRQ* is active-LOW, so I tried both active-LOW and active-HIGH polarities inside DMA2.cpp, but the behavior stayed the same. I also used `svc 'BIHA'` to trigger a sync exception, which verified that the `vector_table` dispatches correctly and the handler does get called; the exception mechanism on the ARM side itself is intact. After checking every layer I could think of, IRQ still never reached the CPU, and I ultimately could not pin down the real cause.

Given the demo deadline, rather than keep drilling into IRQ I chose to wrap things up by polling the `DMA2_CHx_SIZE` register instead. `SIZE` is written back to 0 by `dma_process` after DMA completes, and that action is independent of any ISR, so polling works even when IRQ is not delivered. The instructor's reference code polls `CTRL` on the assumption that an ISR clears it; without an ISR, polling `SIZE` is the behavioral equivalent.

This taught me one thing: a signal being wired correctly does not mean it is actually delivered. The Interfaces tab clearly shows *DMA2\_1.INT0* → *ARM0.nIRQ\_0* with the right polarity per the PSP manual; surface evidence reveals nothing wrong. Only reading the GICDISABLE section tells you Fast Model turns *nIRQ* into a no-op in this state, and even disabling that parameter changed nothing. Surface correctness and behavioral correctness can be separated by a single parameter, sometimes by one you cannot even see.

### 3.1 Symmetric UART Output Across Three Rounds

*UART 0 output*

*UART 1* output

### 3.2 Verifying the RAM2 / RAM3 Content Swap

cpu0 view of  $RAM2 / RAM3$ cpu1 view of  $RAM2 / RAM3$ 

## Closing Thoughts

Two things stuck with me from this project. The first is that the debugging style here is different from what I am used to. My usual flow elsewhere is to write code, run a testbench, look at the waveform, and compare signals, and problems mostly show up at the signal level. In Virtualizer and Fast Model the problems do not always live on the waveform; more often they hide in the gap between spec, manual, and textbook, surfacing only through disassembly, console output, and the small print in the manual.

The second is what it felt like to use a brand-new GUI tool. Virtualizer's complex interface and scattered documentation made the start hard compared to my familiar flow. Maybe more practice would change that judgment, but I cannot tell from this one round alone. Either way, touching a development style this different is interesting in itself.